

Tide retrospective: affine-bilinear specialisation of the Gibbs covariance

Daniel Murfet

7 May 2026

Abstract

This is a 100-line tide that lands the deferred Candidate B of Tide 12 (the 2D pure-quartic refactor): the affine-bilinear specialisation of `gibbsCov`. With the algebra introduced in Tide I2 (`gibbsCov_add`, `gibbsCov_smul`, `gibbsCov_const`; eleven lemmas) freshly available, the headline identity

$$\text{Cov}_t[a_1\varphi + b_1, a_2\psi + b_2] = a_1 \cdot a_2 \cdot \text{Cov}_t[\varphi, \psi]$$

collapses to a thin composition of one-sided affine reductions. The tide exists primarily as the first non-trivial *consumer* of the I2 algebra, validating that the algebra is at the right level of abstraction for downstream multilinearity work. Single session, GPT-5.5 Pro and Claude voted Candidate A on the first round, zero-friction proof, 0 `sorry`.

1 Setting

The 2D pure-quartic tide (1 May 2026) deferred a "Candidate B": an affine-bilinear template for the unperturbed Gibbs covariance. Tide 12's deliberation log records the reasoning: at that point in time, every covariance bash on $\text{Cov}_t[a_1x + b_1, a_2x + b_2]$ -shaped expressions had to unfold `gibbsExpectation` and rewrite by hand, because `gibbsCov` carried no algebra of its own. The right move was to land the algebra first — and only *then* land a thin affine-bilinear corollary as the demonstration that the algebra worked.

The two halves played out as separate tides on 7 May. Tide I2 added the eleven `gibbsCov` algebra lemmas to `Laplace/Gibbs.lean` and `Laplace/Multi/Basic.lean`: symmetry, scalar pulling on each slot (unconditional), constant-collapse on each slot (unconditional), and additivity on each slot (conditional on integrability of the four cross products). With that fresh in cache, the present tide closes Candidate B in a single 100-line file that does almost nothing except chain the algebra together.

The tide is therefore a *narrative-closer* in the strictest sense: it does not prove a new mathematical fact (the affine-bilinear identity is content-free given bilinearity); it proves that the I2 abstraction is load-bearing.

2 The thing formalised

For a 1D Gibbs measure $\exp(-t \cdot L(x)) dx$ on $\mathbb{R}$ and any pair of observables $\varphi, \psi : \mathbb{R} \rightarrow \mathbb{R}$ satisfying the four base integrability conditions on $\exp(-tL)$, $\varphi \cdot \exp(-tL)$, $\psi \cdot \exp(-tL)$, and $\varphi\psi \cdot \exp(-tL)$, the Gibbs covariance $\text{Cov}_t[\varphi, \psi] = \langle \varphi\psi \rangle_t - \langle \varphi \rangle_t \langle \psi \rangle_t$ satisfies the affine-bilinear identity

$$\text{Cov}_t[a_1\varphi + b_1, a_2\psi + b_2] = a_1 \cdot a_2 \cdot \text{Cov}_t[\varphi, \psi] \quad (a_1, b_1, a_2, b_2 \in \mathbb{R}).$$

The constant offsets drop out; the linear scalars factor.

The Lean signature of the headline¹:

```
theorem gibbsCov_affine_affine
  (L : ℝ → ℝ) (t : ℝ) (a1 b1 a2 b2 : ℝ) (φ ψ : ℝ → ℝ)
  (h_exp : Integrable (fun x => Real.exp (-(t * L x))))
  (h_φ : Integrable (fun x => φ x * Real.exp (-(t * L x))))
  (h_ψ : Integrable (fun x => ψ x * Real.exp (-(t * L x))))
  (h_φψ : Integrable (fun x => φ x * ψ x * Real.exp (-(t * L x)))) :
  gibbsCov L t (fun x => a1 * φ x + b1) (fun x => a2 * ψ x + b2)
  = a1 * a2 * gibbsCov L t φ ψ
```

Two one-sided helper theorems (`gibbsCov_affine_left`, `gibbsCov_affine_right`) take the same four hypotheses and reduce a single affine slot to a scalar multiple; the headline is a two-rewrite composition.

3 Proof strategy

The whole proof fits in three rewrite chains, one per theorem.

One-sided affine reduction. For $\text{Cov}_t[a\varphi + b, \psi]$, split via `gibbsCov_add_left` into $\text{Cov}_t[a\varphi, \psi] + \text{Cov}_t[b, \psi]$; the second summand is zero by `gibbsCov_const_left`; the first factors as $a \cdot \text{Cov}_t[\varphi, \psi]$ by `gibbsCov_smul_left`. The four integrability hypotheses `gibbsCov_add_left` requires (on $a\varphi \cdot \exp$, $b \cdot \exp$, $a\varphi\psi \cdot \exp$, $b\psi \cdot \exp$) all derive from the four base hypotheses by `Integrable.const_mul` and a `simp` `[mul_assoc]` cleanup.

The right-mirror is three lines: `gibbsCov_symm` flips the slots, `gibbsCov_affine_left` on the swapped pair runs the same reduction, `gibbsCov_symm` flips back. The only new hypothesis needed is `Integrable(ψφ · exp)`, derived from `Integrable(φψ · exp)` by `simp` `[mul_comm]`.

Headline. For the headline, $\text{Cov}_t[a_1\varphi + b_1, a_2\psi + b_2]$ is reduced by `gibbsCov_affine_left` (with ψ instantiated to the affine combination $a_2\psi + b_2$) to $a_1 \cdot \text{Cov}_t[\varphi, a_2\psi + b_2]$. The right slot still carries an affine combination, but `gibbsCov_affine_right` reduces this directly: $a_2 \cdot \text{Cov}_t[\varphi, \psi]$. Combine with $\leftarrow \text{mul_assoc}$ to fold $a_1 \cdot (a_2 \cdot \cdot) = a_1 a_2 \cdot \cdot$.

The substantive friction in the headline proof is constructing the *mixed* integrability witnesses for the first `gibbsCov_affine_left` application, which requires `Integrable((a2ψ + b2) · exp)` and `Integrable(φ · (a2ψ + b2) · exp)`. Each expands to a sum of two `const_mul` witnesses; this is where GPT-5.5 Pro’s deliberation flag landed (see below).

4 Roadblocks and resolutions

Only one substantive friction, and it was anticipated by GPT-5.5 Pro at deliberation time.

¹In `Laplace/AffineBilinearCov.lean`.

The `Pi.add-vs-single-lambda` mismatch. `Integrable.add` produces `Integrable (f + g)` where `f + g` is `Pi.add` (a sum of two functions, not a single lambda). The first naive attempt to construct `Integrable( $\varphi \cdot (a_2\psi + b_2) \cdot \exp$ )` was

```
have := (h_φψ.const_mul a₂).add (h_φ.const_mul b₂)
simp [add_mul, mul_assoc, mul_add, mul_comm] using this
```

whose target is the canonical single-lambda integrability of $\varphi \cdot (a_2\psi + b_2) \cdot \exp(-tL)$. But `simp` cannot close the gap because the underlying `Pi.add` structure of the hypothesis is opaque to the `simp` set unless we explicitly normalise the lambda first. This is the failure mode flagged in the `laplace CLAUDE.md` from prior tides.

The fix was structural rather than tactical:

```
have eq : (fun x => φ x * (a₂ * ψ x + b₂) * exp(-tL)) =
  (fun x => a₂ * (φ x * ψ x * exp(-tL)) + b₂ * (φ x * exp(-tL))) := by
  funext x; ring
rw [eq]
exact (h_φψ.const_mul a₂).add (h_φ.const_mul b₂)
```

By writing the equality in single-lambda form on both sides (`funext + ring`), the goal after `rw [eq]` is the canonical `Pi.add` that `(h_φψ.const_mul a₂).add (h_φ.const_mul b₂)` delivers verbatim. No `simp` cleanup needed.

Deliberation flag. GPT-5.5 Pro’s response identified exactly this issue at deliberation time: “*if you split an affine side while the other side is still affine, `const_mul` alone is not quite enough for the first mixed witnesses; you also need `Integrable.add` after expanding $(a_1\varphi)(a_2\psi + b_2)\exp$ and $b_1(a_2\psi + b_2)\exp$. After that, the remaining witnesses are just `const_mul + simp/ring`.*” The deliberation also explicitly recommended exposing one-sided affine helpers “*as reusable building blocks*”, which is the structure the implementation took.

The voting converged on the first round: both Claude and GPT-5.5 Pro backed Candidate A (single affine-bilinear theorem with the four base integrability hypotheses); Candidate B (no constants) was deemed redundant with `smul*`, and Candidate C (full bilinearity, four cross terms, sixteen integrability hypotheses) was over-scoped for a narrative-closer tide.

5 What was Mathlib, what was new

Mathlib. `Integrable.const_mul`, `Integrable.add`, the underlying `Pi.add` ordering. Standard ring tactics. No new Mathlib search needed.

New. Three theorems and zero infrastructure (`gibbsCov_affine_left`, `gibbsCov_affine_right`, `gibbsCov_affine_affine`). Everything else — the algebra itself, the integrability lemmas, the Gibbs definitions — was already present from Tide I2 and the earlier seabed.

The tide stays exactly as small as it should be: 101 lines, three theorems, no helpers exported beyond what each theorem visibly needs.

6 Lessons

1. **The `Pi.add-vs-single-lambda` fix is structural, not tactical.** A `funext + ring` equality is more reliable than chained `simp` normalisations for bridging the `Integrable.add` output to a single-lambda target. Worth promoting to the `laplace CLAUDE.md` as the canonical workaround pattern; the existing entry there flags the symptom but not the cleanest fix.
2. **One-sided helpers are nearly free when the proof composes.** The two one-sided affine reductions cost nothing extra beyond what the headline would have needed inline; exposing them widens the public surface area without widening the implementation. GPT-5.5 Pro’s deliberation flag here turned out to be the right architectural call.
3. **The right time to land a thin consumer is right after the infrastructure tide.** Tide I2 landed the algebra; this tide consumed it on a problem that had been deferred specifically for this purpose. Without I2 the present tide would have been a ~ 200 -line bash; *with* I2 freshly cache-warm, it was 100 lines and one round of deliberation. Re-doing the same work *without* the algebra in place would have meant unfolding `gibbsExpectation`, rewriting the products by `integral.add` and `integral.const.mul` hand-rolled, and re-proving constants drop out from scratch — the exact pattern that has appeared in three consumer files (`OneD/Quartic.lean`, `TwoD/PureQuartic.lean`, `TwoD/SemiDegenerate.lean`). The strategic-refactor cleanup on those files (Candidate P1 in the v3 survey) is the natural next narrative beat.

7 Follow-ups

- **Cleanup of the older affine-cov consumer files.** Three files in the `laplace` seabed manually unfold `gibbsExpectation` for affine-cov computations that the new `gibbsCov_affine.*` family makes one-line.² Estimated ~ 150 line net deletion, ~ 200 lines if the cleanup spans the 2D add-separable file too. Process-flavoured but cost-positive.
- **The 2D analogue.** The 2D `gibbsCov` on $(\iota \rightarrow \mathbb{R})$ has a parallel-mirror algebra in the multi-index Gibbs file; a multi-index analogue of the affine-bilinear theorem would take ~ 30 lines and unblock a parallel cleanup of the 2D pure-quartic affine-cov bash. Best done in the same tide as the cleanup if and when that tide is taken.
- **Threepoint-side analogue.** The threepoint `gibbsCov` (perturbed-Gaussian, with $h \cdot A$ in the potential) has no algebra of its own yet; an analogous port (call it “N1” in the v3 survey) of the I2 algebra would make the C1 / G2 parallel sessions’ kappa-3 strict-improvements collapse from ~ 50 -line bashes to ~ 5 -line specialisations. The two parallel sessions on C1 / G2 currently in flight will surface whether the port is needed.
- **Promote `gibbsCov_affine.*` to the lemmas section of `Laplace/Gibbs.lean`.** If the cleanup tide above lands, the helpers earn enough consumer sites that they belong with the `gibbsCov.*` algebra rather than in their own file.

²The 1D quartic and the 2D pure-quartic and semi-degenerate consumer-site bashes; pointers in the v3 survey.